

Topic 5 Review [99 marks]

1. [Maximum mark: 15]

25M.1.HL.TZ2.12

(a) Consider the following recursive function:

```
recfn(N)
  if N <= 0
    then
      return 0
    else
      if N mod 2 = 0
        then
          return recfn(N-1) - N
        else
          return recfn(N-1) + N
      endif
    endif
  end recfn
```

Determine the value of variable X after execution of the following program statement:

$X = \text{recfn}(5)$

Show all your working.

[5]

Markscheme

*Award **max** [5].*

*Award [1] for each correct recursive call shown, **x5**.*

Award [1] for the correct result.

$X = \text{recfn}(5) = \text{recfn}(4) + 5$;
 $= \text{recfn}(3) - 4 + 5 = \text{recfn}(3) + 1$;
 $= \text{recfn}(2) + 3 - 4 + 5 = \text{recfn}(2) + 4$;
 $= \text{recfn}(1) - 2 + 3 - 4 + 5 = \text{recfn}(1) + 2$;
 $= \text{recfn}(0) + 1 - 2 + 3 - 4 + 5 = \text{recfn}(0) + 3$;
 $= 0 + 1 - 2 + 3 - 4 + 5 = 0 + 3 = \mathbf{3}$;

Note: *The working may be shown differently.*

Examiners report

While many of the responses scored full marks, inaccuracies in handling + and – signs in the recursion task resulted in students not getting the maximum marks.

(b) Describe **two** basic stack operations.

[4]

Markscheme

*Award **max** [4].*

*Award [1] for name and [1] for description of operation, **x2**.*

Push;

Add an element to the top of a stack;

Pop;

Remove an element from the top of a stack;

IsEmpty;

Check if the stack contains no elements/returns True when a stack is empty, returns False otherwise;

IsFull;

Check if the stack has reached maximum capacity/returns True when a stack is full, returns False otherwise;

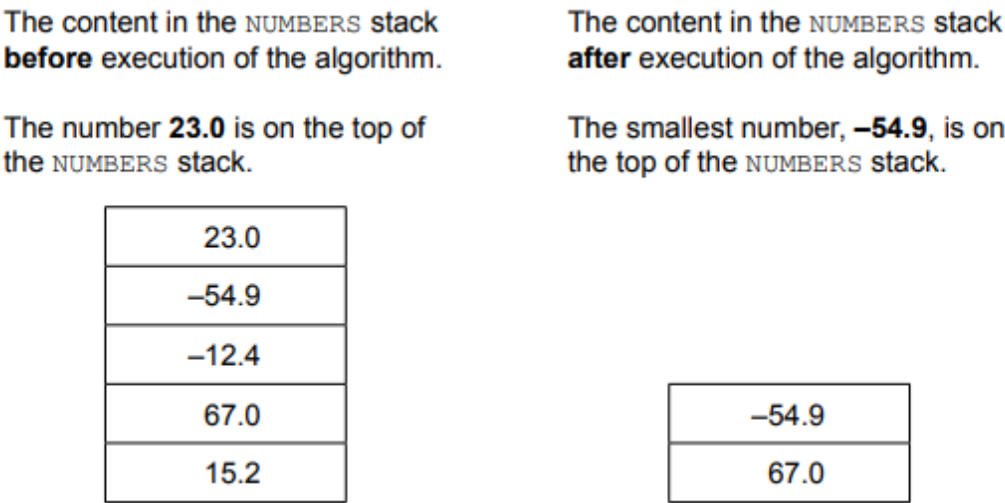
Examiners report

The responses stated push and pop operations with correct explanations, some students did not reference the 'top' of the stack, indicating a gap in

precise technical terminology.

The **NUMBERS** stack stores real numbers. An algorithm should be constructed that will find the smallest number and the largest number stored in the **NUMBERS** stack. After execution of the algorithm, **only** these two numbers should be stored in the **NUMBERS** stack (see **Figure 1**).

Figure 1: Example data stored in the **NUMBERS stack before and after execution of the requested algorithm**



(c) Construct this algorithm in pseudocode.

You may assume that the **NUMBERS** stack already contains real numbers.

You must use the access methods of a stack.

[6]

Markscheme
Award max [6]. Award [1] for correct initialization (before the loop) Award [1] for correct loop through the NUMBERS stack using <code>isEmpty()</code> Award [1] for taking an element from the top of the NUMBERS stack using <code>pop()</code> (within the loop)

Award [1] for changing the value of SMALLEST if needed

Award [1] for changing the value of LARGEST if needed

Award [1] for placing SMALLEST and LARGEST on the stack NUMBERS using push() (after the loop)

Award [1] for the order of push operations: first push LARGEST onto the stack then SMALLEST (so that SMALLEST will be on the top of stack NUMBERS)

Example 1:

```
X = NUMBERS.pop()
SMALLEST = X
LARGEST = X
// OR
// initialization
// SMALLEST= 999999999999 (any very large number)
// LARGEST= -999999999999 (negative number!)
loop while not NUMBERS.isEmpty()
    //searching for the smallest and largest value
    X=NUMBERS.pop()
    if X <= SMALLEST
        then SMALLEST = X
    else if X >= LARGEST
        then LARGEST = X
    end if
end if
end loop
NUMBERS.push(LARGEST)
NUMBERS.push(SMALLEST)
```

Example 2:

```
SMALLEST = NUMBERS.peek() //initialization
LARGEST = SMALLEST
loop while NUMBERS.isEmpty()= false
    //searching for the smallest and largest value
    X=NUMBERS.pop()
    if X < SMALLEST
        then SMALLEST = X
    end if
    if X > LARGEST
```

```
        then LARGEST = X
      end if
    end loop
    NUMBERS.push(LARGEST)
    NUMBERS.push(SMALLEST)
```

Examiners report

Many students have written the pseudocode algorithm correctly. Some students have mixed up stack operations with general array access techniques, suggesting the need for reinforcement of their distinct characteristics.

2. [Maximum mark: 4]

24N.1.HL.TZ0.7

Consider the following recursive method:

```
rec(A,B)
  if B=0
    then
      return A
    else
      return rec(B,A mod B)
    end if
  end rec
```

Determine the value of $\text{rec}(20,12)$. Show all your working.

[4]

Markscheme

*Award **[4 max]***

The work out may be differently presented.

```
rec(20,12) = rec (12, 20 mod 12) = rec (12, 8) ;  
           = rec (8, 12 mod 8) = rec (8, 4)  ;  
           = rec (4, 8 mod 4) = rec (4, 0)   ;  
           = 4                               ;
```

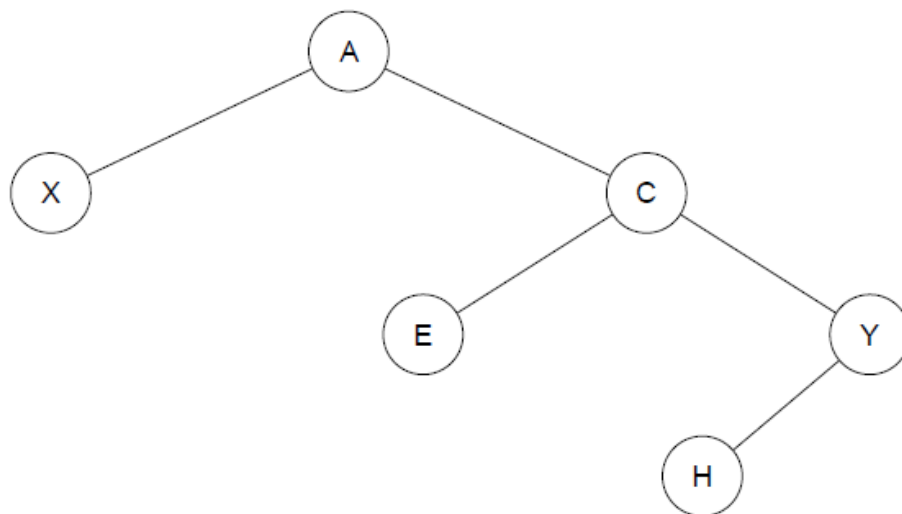
Examiners report

A simple enough recursive algorithm, leading to a reasonably high number of correct answers.

3. [Maximum mark: 2]

24N.1.HL.TZ0.9

Consider the following binary tree:



(a) State the result of postorder traversal of the binary tree.

[1]

Markscheme

Award [1 max]

X E H Y C A;

Examiners report

Students mostly gave good responses for this question. Some students confused postorder traversal with preorder traversal.

(b) State the result of inorder traversal of the binary tree.

[1]

Markscheme

Award [1 max]

X A E C H Y;

4. [Maximum mark: 4]

24N.1.HL.TZ0.10

Outline **two** operations on a queue data structure.

[4]

Markscheme

Award [4 max]

Award [1] for naming a queue operation and award [1] for defining/describing the operation, x2.

Enqueue;

adds (stores) an item to(at) the back of the queue;

Dequeue;

removes (and returns) an item from the front of the queue;

isEmpty;

is used to check if the queue contains any elements or not/ returns True if the queue is empty, False if the queue is not empty;

isFull;
is used to check if the queue has reached its maximum capacity/ returns
True if the queue is full, returns False if it is not full;

front/peek;
returns the value of the first item without deleting it;

Examiners report

This question had a good number of correct responses. Reasonably well
answered with many gaining at least two of the marking points.

5. [Maximum mark: 15] 24N.1.HL.TZ0.13

Cell phones are used to make phone calls and send text messages. They can also
browse the internet and run apps.

An incoming call will interrupt an app being executed.

- (a) Describe how a stack data structure can be used to resume the
app that was initially running.

[2]

Markscheme

Award [2 max]

all parameters from the application will be pushed onto a stack;
and popped off back (into the CPU) when the call is over;

when the incoming call arrives, the OS saves the current state of the app
and pushes it onto the stack;

once the call is ended the OS pops the most recent state (the state of the app right before the call) from the stack and restores it;

Examiners report

A wide variety of answers and quality of answers was seen for this question covering the full mark range. Students often lost marks by not providing answers with sufficient clarity as required to achieve all the marks for a particular question.

A few students did not attempt this question. However, those who did generally answered well. A few students incorrectly used enqueue/dequeue operation on the stack.

- (b) Describe **one** way a cell phone app can help the user improve their health.

[2]

Markscheme

Award [2 max]

Award [1] for identifying a way and award [1] for an expansion.

Different health cell phone apps can track/record data;
To detect unhealthy conditions;

Note: *Accept examples of cell phone apps that can monitor any vital signs and be used in diagnostic, such as sleep tracking functionality in prenatal care, etc.*

Cell phone apps can access nutrition information/guide;
So, users can eat healthier food;

Cell phone apps to monitor/track exercise (track distance, time, speed, and calories burned);
And serve a role as motivator to do more of it;

Note: *Accept other reasonable answers.*

Examiners report

Answers to this question vary from poor to excellent.

- (c) Outline why a higher screen resolution leads to higher quality images on a cell phone screen.

[2]

Markscheme

Award [2 max]

by increasing screen resolution more elements can be addressed on the same area of screen;

so, more data from an image can be displayed (in the same area);

higher screen resolution means finer colour transition/ better colour gradients;

especially in areas where colours shift gradually;

higher screen resolution increases text clarity;

so even smaller font sizes can be read as the letters appear better defined;

higher resolutions indicate more pixels;

so, more details can be captured;

Examiners report

This was answered reasonably well.

(d.i) Explain what is meant by GPS (global positioning system).

[4]

Markscheme

Award [4 max]

The Global Positioning System (GPS) is a navigation system/ a system used to determine the ground position of an object;

It consists of three segments: satellites, control (made up of Earth-based monitor stations), and GPS receivers (user equipment);

A number of satellites continuously send out signals containing their position and the exact time the signal was sent;

GPS receiver (*accept mobile phones*) picks up the signal and determines the distance to each satellite (by calculating the time required to receive the signal);

By using trilateration, it determines the exact location (by knowing the distance to at least four satellites);

Examiners report

The question was reasonably well answered with many gaining at least 2 or 3 of the marking points. Most students knew that the Global Positioning System (GPS) is a system used to determine the ground position of an object, and that it works by communication with satellites.

Some students failed to achieve full marks as result of incomplete and vague descriptions. There are students who incorrectly wrote that satellites

calculate position of an object. "Satellites draw maps" is also one example of incorrect students' answers that frequently appeared.

- (d.ii) Outline **one** concern of cell phone apps sharing the location of the user.

[2]

Markscheme

Award [2 max]

Award [1] for a concern and [1] for an expansion.

Privacy concerns;

All the time and at all locations/ in a place and time when a user has a reasonable expectation of privacy;

Safety concerns;

surveillance is possible/ possible sharing the user's whereabouts with stalkers;

Advertisers could profile;

User's physical/online habits;

Erroneous data could be created;

Because of missing/incorrect location information;

- (e) Explain **one** advantage of using a dedicated operating system on a cell phone instead of a generic operating system.

[3]

Markscheme

Award [3 max]

A dedicated operating system for a mobile phone will take up less storage space than a full-sized operating system;
this will allow the device to function more quickly;
because it doesn't contain features that aren't needed;

The operating system is optimized for speed/ made to be as fast as possible;
uses resources effectively;
because it is created for a specific hardware;

Security is improved;
by prohibiting services that would run on a generic operating system/
prohibiting software update;
only allowing very specific range of IP addresses or keys allowed;

To suit a particular individual or task / customization;
only a minimal/ basic modification can be made;
in user interface/ the network connections/ and virtually anything else;

A dedicated OS is designed specifically for a task;
makes the most efficient use of hardware available;
does not include unnecessary features;

Note: Accept other reasonable answers on customization/ security/ speed/ optimization.

Examiners report

The question was well answered with many gaining full marks.

6. [Maximum mark: 15]

24N.1.HL.TZ0.14

An application requires a list of students' names to be held in alphabetical order, from A to Z, in the primary memory.

The application must allow for insertion and deletion of names.

- (a) Outline **two** advantages of using a linked list rather than a static array in this scenario.

[4]

Markscheme

Award [4 max]

Award [1] for an advantage and [1] mark for an expansion, x2.

Names can be inserted/deleted quickly/easy just by manipulating the pointers/ by updating the address;
without resorting/shuffling (as in an array);

Better use of memory/ the number of names (size of the list) is not fixed (as in an array);
so, no wasted space / linked list can shrink and grow at the runtime (by deallocating or allocating memory);

No need for an initial size/ predetermined size in linked list (as in an array);
which allocate space that may or may not be fully in use;

Examiners report

A few students did not attempt this question. However, those who did generally answered well.

- (b) Explain how a specified student's name could be found in a linked list.

[5]

Markscheme

Award [5 max]

Temporary pointer should be used;
and initially set to point to the beginning of the list;
loop through the list;
follow the internal pointers;
compare the name in the node pointed to by the temporary pointer with
the searched name;
If they are same (the name is found) then stop searching;
If the end of the list is reached / temporary pointer equals null then the
searched name is not on the list;

Examiners report

Students were required to explain how a specified student's name could be found in a linked list. Students who gave a list of logical steps written in the style of an algorithm, rather than as a purely descriptive answer, achieved higher marks.

A double linked list would be preferred over a single linked list for applications that require the ability to move forward (from A to Z) and backward (from Z to A) through the list of names.

(c.i) Describe the structure of a double linked list.

[4]

Markscheme

Award [4 max]

Double linked list DLL has two external pointers, one pointer points to the first element of the DLL, and other points to the last element in the DLL;
Each node consists of a data field(s);
and two pointers (for example, called NEXT and PREV);
Each node is linked with its next node (using its pointer NEXT) and previous node (using its pointer PREV);

The NEXT pointer of the last node is null (to mark the end of the list - forward traversal) and the PREV pointer of the first node is null (to mark the end of list - backward traversal);

Note: Accept diagrams.

Examiners report

The question was reasonably well answered with many gaining at least 2 or 3 of the marking points.

(c.ii) State **two** applications of double linked lists in computing.

[2]

Markscheme

Award [2 max]

Storing the browsing history which allow going forward and backwards in browsing history;

A music player which has next and previous buttons;

In LRU caching/ cache that is employed in cases which uses most/least recently accessed items/ examples like in the case of android phone home screen to save the most recently used apps;

An application that has a 'Most Recently Used' element / for example, list of file names;

In games, to represent anything that requires an order in a game / A DLL can be used to represent a deck of cards;

Undo-Redo functionality (for example, in word processing);

Note: Accept any application where front and back navigation is required.

7. [Maximum mark: 15]

24N.1.HL.TZ0.15

The two-dimensional array **MAT** is used to store randomly generated integers in the range from 0 to 9.

In the **MAT** array, some numbers can be repeated multiple times and other numbers may not appear at all (see **Figure 1**).

Figure 1: Example data stored in the MAT array

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	0	2	5	9	3	6	1
[1]	0	2	9	0	7	0	0
[2]	4	2	7	9	5	7	1
[3]	7	2	3	1	5	4	0
[4]	3	6	3	9	9	6	0
[5]	7	4	3	1	7	9	3
[6]	3	9	6	4	2	2	4

The one-dimensional **COUNT** array will show how many times each of the randomly generated numbers is found in the **MAT** array (see **Figure 2**).

Figure 2: The content of the COUNT array

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
7	4	6	7	5	3	4	6	0	7

Figure 2 shows that number 4 occurs 5 times (**COUNT**[4]=5) in the **MAT** array.

- (a) Construct an algorithm in pseudocode to create the **COUNT** array as described. You may assume that the **MAT** array is created and the **COUNT** array is initialized with zero values.

[4]

Markscheme

Award [4 max]

Example 1:

Award [1] for the row loop

Award [1] for the column loop

Award [1] for the correct index (index in the COUNT array is MAT[R][C])

Award [1] for incrementing the correct element in the COUNT array

```
loop R from 0 to 6 //accept 6 or MAT.length-1
  loop C from 0 to 6
    COUNT[MAT[R][C]] = COUNT[MAT[R][C]] + 1
  end loop
end loop
```

Example 2:

Award [4 max] as follows:

Award [1] for the outer loop

Award [1] for the inner loop

Award [2] for correct if statement (comparing MAT[R][C] with all values in the range from 0 to 9), award [1] for a minor error

Award [1] for incrementing the correct element in the COUNT array

```
loop R from 0 to 6
  loop C from 0 to 6
    if MAT[R][C] = 0 then COUNT[0] = COUNT[0]+1
    else if MAT[R][C] = 1 then COUNT[1] = COUNT[1]+1
    else if MAT[R][C] = 2 then COUNT[2]= COUNT[2]+1
    else if MAT[R][C] = 3 then COUNT[3]= COUNT[3]+1
    else if MAT[R][C] = 4 then COUNT[4]= COUNT[4]+1
    else if MAT[R][C] = 5 then COUNT[5]= COUNT[5]+1
    else if MAT[R][C] = 6 then COUNT[6]=C OUNT[6]+1
    else if MAT[R][C] = 7 then COUNT[7]= COUNT[7]+1
    else if MAT[R][C] = 8 then COUNT[8]= COUNT[8]+1
```

```

        else if MAT[R][C] = 9 then COUNT[9]= COUNT[9]+1
        end if
    end if
end if
end if
end if
end if
end if
end if
end if
end if
end loop
end loop

```

Examiners report

Answers to this question vary from poor to excellent. Some students constructed excellent algorithms, and they gained full marks on this question.

Some good algorithms were seen for this question. Some students made mistakes in defining the loop limits and array indexes.

In **Figure 2**, COUNT[8]=0 because the number 8 does not appear in the MAT array.

- (b) Construct an algorithm in pseudocode that uses the COUNT array to output the numbers that do not appear in the MAT array. If all of the numbers from 0 to 9 are present, then it will output an appropriate message.

[5]

Markscheme

Award [5 max]

Award [1] for using a flag

Award [1] for a correct loop (for, while, loop until)

Award [1] for the correct condition in if statement and changing the flag (if needed)

Award [1] for outputting all numbers that do not appear in MAT

Award [1] for checking the flag (if statement) after the loop

Award [1] for outputting an appropriate message

F=0

loop K from 0 to 9 //accept 9 or COUNT.length-1

 if COUNT[K]=0

 then

 F = 1

 output(K)

 end if

end loop

if F = 0

 then output ('all the numbers from 0 to 9 are present ')

end if

Examiners report

A range of performance was seen here with many scoring full marks too.

The mode is the number that appears most often in the MAT array. There can be more than one mode.

0, 3 and 9 are the modes of the numbers stored in the MAT array (see **Figure 2**).

- (c) Construct an algorithm in pseudocode that uses the COUNT array to output the mode(s) of the numbers stored in the MAT array.

Markscheme

Award [6 max]

Example 1:

Award [1] for initializing HV

Award [1] for a correct loop

Award [1] for comparing COUNT[K] with HV

Award [1] for changing the value of HV (if needed)

Award [1] for the second loop (loop after the first one)

Award [1] for checking if HV is equal to COUNT[K]

Award [1] for outputting values that occur most frequently

//searches for the highest value in the COUNT array

HV=0 // or HV = COUNT[0]

loop K from 0 to 9

 if COUNT[K] > HV

 then HV = COUNT[K]

 end if

end loop

//outputs the mode(s)

loop K from 0 to 9

 if COUNT[K] = HV

 then output(K)

 end if

end loop

Example 2:

Award [1] for initializing MAX

Award [1] for a correct loop

Award [1] for comparing COUNT[K] with MAX and changing the value of MODE_IND[K] (if they are equal)

Award [1] for comparing COUNT[K] with MAX and changing the value of MAX (if COUNT[K] is greater than

```

MAX)
Award [1] for reinitializing the MODE_IND array (the loop within if statement)
Award [1] for assignment MODE_IND[K]=1
Award [1] for outputting modes

//assume MODE_IND is initialized
//an integer array zero array in this example, a Boolean array could
be
//used instead
//the MODE_IND array is used to flag modes

MAX = 0 // MAX = COUNT[0]
loop K from 0 to 9
  if COUNT[K] == MAX
    then MODE_IND[K] = 1 //flags mode K
  endif
  if COUNT[K] > MAX
    then
      MAX = COUNT[K] //the new largest value
      loop I from 0 to 9
        MODE_IND[I] = 0 //reinitializing the array
      end loop
      MODE_IND[K] = 1 //flags the new mode K
    end if
  end loop

loop K from 0 to 9 // outputs modes
  if MODE_IND[K] == 1
    then output(K)
  end if
end loop

```

Examiners report

Many students constructed excellent algorithms. However, constructing algorithms proved difficult for some students.

Some students did not even attempt this question.

8. [Maximum mark: 2]
Identify **two** applications of a stack.

24M.1.HL.TZ1.8

[2]

Markscheme

Award [2 max]

Running recursive processes;
Holding return memory addresses / function call;
Expression evaluation and conversion/ evaluating arithmetic expressions;
String reversal;
Interrupt handling;
Undo / redo in gaming/any other example;
Backwards navigation on Web browser;
Backtracking path in algorithms;
Stack based parsing algorithms;
Reversing a queue;
Depth first search algorithms;

Examiners report

Applications of stack had to be computer related and not generic like 'pile of dishes'. Responses like 'Undo' without the mention of an application where 'Undo' is used were not credited.

9. [Maximum mark: 15]
Linked lists and binary trees are examples of dynamic data structures.

24M.1.HL.TZ1.15

(a.i) Outline **one** benefit of using dynamic data structures.

[2]

Markscheme

Award [2 max]

Award [1] for the benefit and award [1] for expansion.

Efficient use of memory;
memory is allocated during the execution of the program as needed /
memory is deallocated and re-used when no longer required;

There is no need to predefine the size of a dynamic data structure;
so, they can grow/data can be added or shrink in size/data removed
depending on the requirements;

Efficient insertion and deletion operations can be carried out;
with the adjustment of pointers/ no need to shift elements / as they are
stored in chained memory locations;

Examiners report

Many correct answers were given here, 'Efficient use of memory' being the most common one. Students who went on to explain it further scored a second mark.

(a.ii) Outline **one** drawback of using dynamic data structures.

[2]

Markscheme

Award [2 max]

Award [1] for drawback and award [1] for further expansion.

(Memory) overflow;

Can happen if more memory is actually used than has been allocated/available;

(Memory) underflow;

can happen if attempting to delete an element from an empty data structure;

Harder to program;

because it is more difficult to keep track of the size and location of the data structure;

Potential for memory leaks;

when allocated memory is not properly deallocated after use;

Errors, crashes & security vulnerabilities;

due to null pointer dereferencing, dangling pointers, or memory access violations;

More memory usage for the set amount of data;

as dynamic data structures require additional memory to store pointers;

Slower execution time/longer to traverse etc;

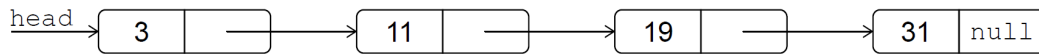
due to more programming statements required/ manipulation of pointers;

Examiners report

A large number of students mentioned the drawback of dynamic data structure by making a comparison to the static data structure. This was not a part of the question and should have been answered accordingly. Here too there were many vague answers such as 'less time consuming' or 'It is slower'.

Figure 3 represents a linked list containing the numbers 3, 11, 19 and 31.

Figure 3: A linked list



- (b) Describe the steps to find and delete the node containing data item 19.

[5]

Markscheme

Award [5 max]

Initialise current pointer to head/start of linked list;
check if data item is the desired value/19;
If not, store the value of current pointer in previous pointer;
then change the current pointer to next position in linked list;
repeat until data item 19 is found (or until end of list reached);
if data item 19 found, change the next (pointer) of the previous pointer to point to the following node / node containing 31
clear the memory for the deleted node / node containing 19;

Alternate answer:

Check the data item at head to see if it is the desired value/19;
if not, initialise a current pointer to point to next node in the linked list;
check the next item in the linked list;
repeat until data item 19 is found (or until end of list reached);
if data item 19 found, change the next of the current pointer to point to node after the next node ($\text{current.next} = \text{current.next.next}$) / node containing 31;
clear the memory for the deleted node / node containing 19;

Examiners report

Very few students were able to attempt this question successfully by making specific reference to the use of pointers and their manipulation. There is also a need for care in the use of the terminology here. Students

who went on to describe a linear search without any reference to the pointers did not gain marks.

(c) Describe the structure of a node in a binary tree.

[2]

Markscheme

Award [2 max]

A node contains a data element;

A node has two pointers (a pointer to a left child (node) and a pointer to the right child (node));

Examiners report

Many correct responses were seen here. Those who lost marks here described the structure of a 'binary tree' instead of the 'node'.

These numbers are input in the following order:

17 11 23 5 3 19 37 31

and inserted in a binary tree such that an inorder traversal of the binary tree outputs the numbers sorted in ascending order.

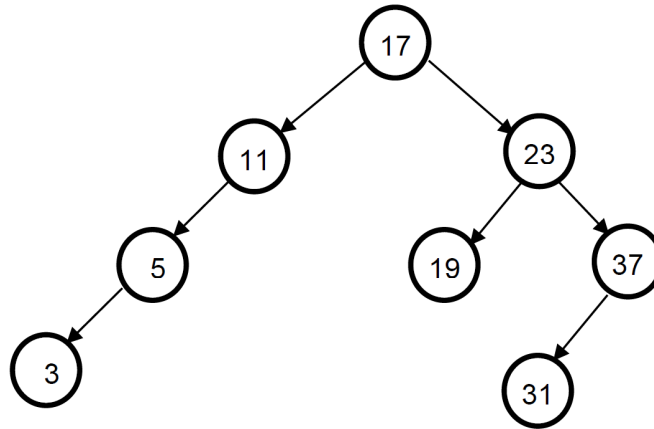
(d.i) Sketch the resulting binary tree.

[3]

Markscheme

Award [3 max]

Root node correct;
Left subtree correct;
Right subtree correct;



Examiners report

A few students were able to score all three marks here while others identified at least the root node correctly.

(d.ii) State all the leaf nodes in the binary tree sketched in part (d)(i).

[1]

Markscheme

Award [1 max]

3, 19, 31

*Candidate's answer **must** match their answer to part (d)(i).
Allow follow through from part (d)(i).*

Examiners report

Most of the students were able to identify the leaf nodes despite incorrectly answering part (i).

10. [Maximum mark: 2]

23M.1.HL.TZ1.2

Identify **two** applications of queues in computing.

[2]

Markscheme

Award [2 max]

print queue (serving requests on a shared printer) / spooling in printer;
CPU task scheduling;
handling of interrupts (in the same order as they arrive);
buffer for devices like keyboard;
queues in routers/ switches;
mail queues;
simulation/ computer modelling of physical queues (e.g., a customer waiting line in a supermarket queue, a call centre where technical personnel take calls and provide service, etc.);
handling website traffic/ network congestion;
maintaining playlist in media player;

Note: Accept other appropriate examples of applications of queues in computing.

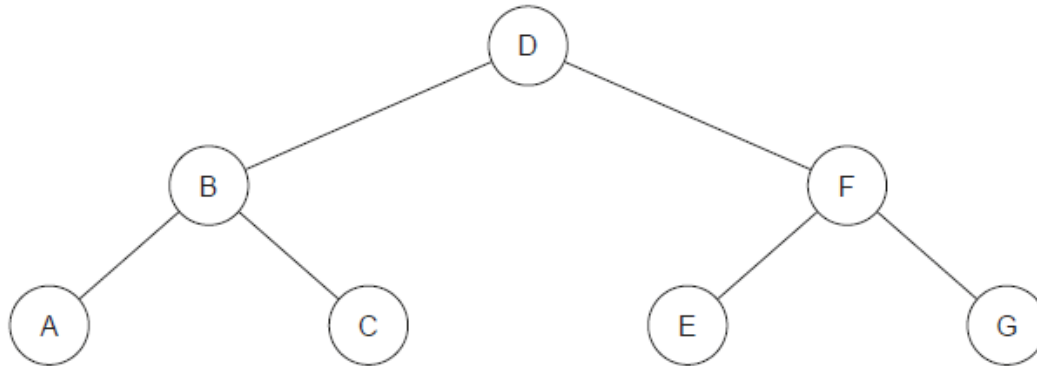
Examiners report

Most candidates were able to identify two applications of queues in computing.

11. [Maximum mark: 5]

23M.1.HL.TZ1.8

Consider the following binary tree, in which each node stores a value greater than all the values in the node's left subtree and less than those in its right subtree.



(a) Identify the leaf nodes in this binary tree.

[1]

Markscheme

Award [1 max]

A C E G;

Examiners report

It is evident that some candidates do not know the meaning of the term 'leaf node'.

(b) State the result of the postorder traversal.

[1]

Markscheme

Award [1 max]

A C B E G F D;

Examiners report

A well answered question.

- (c) Sketch the resulting binary tree after the deletion of the root node.

[3]

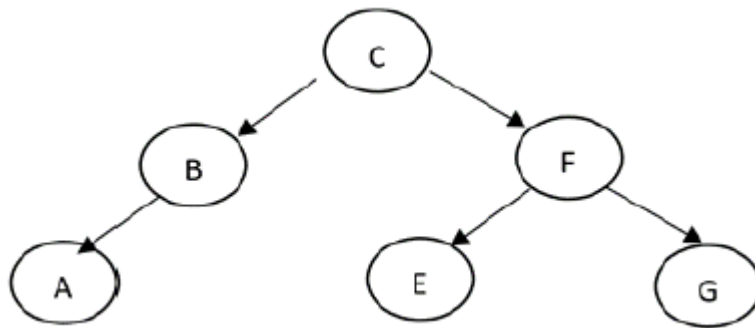
Markscheme

Award [3 max]

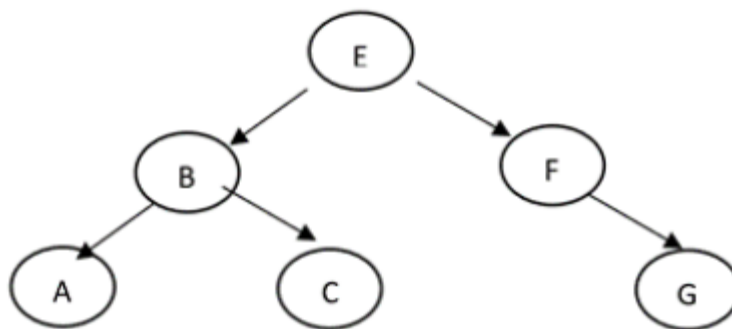
Award [1] for the correct root

Award [1] for the correct left subtree

Award [1] for the correct right sub-tree.



OR



Examiners report

Some candidates did not delete the root node.

12. [Maximum mark: 3]

22N.1.HL.TZ0.8

A computer science student is coding and running a program while several documents, such as essays, lab reports and homework, are being printed out.

(a) Define the term *queue* as a data structure.

[1]

Markscheme

Award [1] max.

A data structure in which items are added to the tail/rear (one end) and removed from the head/front (another end) / first in first out (FIFO) data structure;

Examiners report

A well answered question.

(b) Identify **two** different queues that are used in this scenario.

[2]

Markscheme

Award [2] max.

Print queue;

Keyboard queue;

Accept any other job queue (program is running!);

Examiners report

A well answered question.

13. [Maximum mark: 2]

22M.1.HL.TZ0.9

(a) Identify **one** characteristic of a queue.

[1]

Markscheme

Award [1 max]

A linear (abstract) data structure;

First in First out/FIFO;

Elements can only be added at one end(rear/tail) and removed from the other (front/head);

Once a new element is inserted into the queue (enqueue), all the elements inserted before the new element in the queue must be removed (dequeue), to remove the new element;

Examiners report

Many candidates correctly answered this question.

(b) Identify **one** application of a queue.

[1]

Markscheme

Award [1 max]

Print queue;

CPU task scheduling;

Simulation of a real-life scenario (call centre, supermarket queue);

Playlist queue;

Interrupt queues;

Queues in routers and switches;

Mail queues;

Buffers in devices like keyboard;
Website traffic;

Note to examiners: Reward other reasonable responses.

Examiners report

Many candidates correctly answered this question.